

D NUMERICAL SOLUTION

D.1 Standard form

Setting $X_i = (u, u', v, v', w, p)$ and $F_i = (0, f_1, 0, f_2, 0, f_3)$, the equations can be written in the compact tensor form:

$$\frac{\partial X_i}{\partial z} = A_{ij}X_j + F_i. \quad (\text{D.1})$$

We have a set of linear differential equations with boundary conditions both at the roughness height ($z = z_0$) and at the boundary layer height ($z = z_i$). It may sound as an easy problem, but it is not. The precision required for small values of kz_0 (relevant for offshore wind farms with long wakes) discards shooting methods usually used for Boundary Value Problems (BVPs). To solve this, Fuga used a modified version of the *chasing method* for linear systems, first proposed by Berezin and Zhidkov (1965).

D.2 The chasing method

The chasing method first transforms boundary conditions at one boundary to equivalent boundary conditions at the other boundary. With all boundary conditions at the same boundary, we have an initial value problem, which can be integrated using e.g. Runge-Kutta. The transformation works like this: we have a boundary ¹ at z_1 (equal to either z_0 or z_i):

$$b^1 = y_i^1 * X_i(z_1), \quad (\text{D.2})$$

and wish to find an equivalent condition:

$$b(z) = y_i^*(z) X_i(z), \quad (\text{D.3})$$

at some other point z . The asterisk denotes complex conjugation (making the right-hand side a proper inner product of two vectors). The trick is first to solve the following initial value problem:

$$\frac{\partial y_i}{\partial z} = M_{ij} y_j \quad (\text{D.4})$$

$$y_i(z_0) = y_i^1, \quad (\text{D.5})$$

where $M = -A^\dagger$, and the dagger denotes the Hermitian adjoint (complex conjugate transpose, i.e. $M_{ij} = -A_{ji}^*$). This is a good choice, since differentiation of both sides of (D.3) with respect to z leads, using (D.1), to a simple equation, where terms involving X_i cancel, i.e.

$$\frac{\partial b(z)}{\partial z} = \frac{\partial (y_i^* X_i)}{\partial z} = \frac{\partial y_j^*}{\partial z} X_j + y_i^* \frac{\partial X_i}{\partial z} = (M_{ji} y_i)^* X_j + y_i^* A_{ij} X_j + y_i^* F_i = -y_i^* A_{ij} X_j + y_i^* A_{ij} X_j + y_i^* F_i = y_i^* F_i. \quad (\text{D.6})$$

On integration, we find that

$$\Delta b(z) \equiv b(z) - b(z_0) = \int_{z_1}^z y_i^*(z') F_i(z') dz'. \quad (\text{D.7})$$

The integral can be evaluated numerically in parallel with the solution of (D.4), which can be done without any knowledge of $X(z)$ or $b(z_0)$. In this way the three lower boundary conditions can be transformed into three upper boundary conditions, and the problem can be integrated down from z_i to z_0 as an ordinary initial value problem. This method, called ‘chasing’, is used by Mathematica. Actually there is no need for the second integration, if three more independent lower boundary are added. Even if $b(z_0)$ is unknown we can still evaluate $\Delta b(z)$ and $y(z)$ for these conditions. Next step is to decide where to place z_i . Once this has been decided, we have 6 conditions at the upper boundary: 3 conditions equivalent to the lower boundary conditions plus 3 upper boundary conditions. From these we can determine $X(z_i)$ and evaluate $b(z_i) = y_j^*(z_i) X_j(z_i)$ for all 6 conditions. But then we have 6 conditions at all levels because $b(z_0) = b(z_i) - \Delta b(z_i)$ is now known, and we can finally find $X(z)$ at all levels of interest.

¹We are only considering *linear* conditions here.

Unfortunately, the chasing method often fails because the 6 conditions used to determine $b(z_i)$ turn out to be numerically ill-conditioned. The problem is lack of numerical precision. As the lower boundary conditions are transported upwards, the three corresponding y 's tend to be more and more parallel to each other, and extremely high numerical precision is needed to tell them apart. Investigations using Mathematica indicate that as many as 60 significant digits are sometimes needed, which is about twice as many as are available using quadruple precision floating-point reals. This 'stiffness' of the system of equations is caused by solutions to (D.4) that grow extremely fast (exponentially or even faster). They exist for both upward and downward integration. All solutions, therefore, tend to become parallel to the fastest growing solution. In this way the lower boundary conditions effectively end up as upper boundary condition that are either not linearly independent or contradicting each other.

A modified version of the chasing method was therefore used. The idea is to reformulate the transported conditions as often as needed in order to separate them before running out of numerical precision. The integration starts from below at $z = z_0$, where we have the three boundary conditions $X_1 = X_3 = X_5 = 0$ (i.e. $u = v = w = 0$), each with a corresponding y_i and b_i (all three equal to 0 at z_0). It is practical to use these y 's as the first (i.e. leftmost) three columns in a 6×6 matrix $Y_{ij}(z)$. The remaining three columns represent three additional conditions. The initial values should be chosen so that $Y_{ij}(z_0)$ is regular. We choose to define them in such a way as to take the initial value $Y_0 \equiv Y(z_0)$ to be *unitary*, e.g.

$$Y_{ij}(z_0) = \begin{bmatrix} \cos \beta & -\sin \beta & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \cos \beta & -\sin \beta & 0 \\ \sin \beta & \cos \beta & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \sin \beta & \cos \beta & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \quad (\text{D.8})$$

Here β is the angle between the x -axis and the wave vector $\mathbf{k}$, but any angle would do since $u \cos \beta + v \sin \beta = -u \sin \beta + v \cos \beta = 0$ is just another way of saying $u = v = 0$. The reason for focusing on these two particular horizontal wind components is that their governing equations are quite different, so it might be a good idea to keep them separate. We only have $b(z_0)$ values for the first three columns, but according to eq. (D.7) we can find Δb_i for all six values of i .

We now rewrite eq. (D.4) in matrix form (which does not mix the columns of Y_{ij})

$$\begin{aligned} \frac{\partial \mathbf{Y}}{\partial z} &= \mathbf{M} \mathbf{Y} \\ \mathbf{Y}(z_0) &= \mathbf{Y}_0 \\ \frac{\partial \Delta \mathbf{b}}{\partial z} &= \mathbf{Y}^\dagger \mathbf{F} \\ \Delta \mathbf{b}(z_0) &= 0 \end{aligned} \quad (\text{D.9})$$

The forcing F_j can be divided into three components: (i) longitudinal forcing acting in a direction opposite to the (mean) wind direction, (ii) transverse forcing (relevant for yawed rotors), and (iii) vertical forcing (relevant for tilted rotors). Taking advantage of the linearisation of the problem, it is appropriate to derive three separate solutions, which later can be scaled and superimposed to give the response of the full three-dimensional forcing. Therefore, there will be a separate solution for each of the three choices:

$$\begin{aligned} F_j(z) &= f_1(z) \delta_{j2} \\ F_j(z) &= f_2(z) \delta_{j4} \\ F_j(z) &= f_3(z) \delta_{j6}. \end{aligned} \quad (\text{D.10})$$

This means that there will be three sets of b_i and Y_{ij} – one for each of the three types of forcing. In principle, $f_j(z)$ can be any function of z , but we will, for convenience, restrict $f_j(z)$ to be piece-wise linear. To this end, we define a set of levels

$$z_j = z_0 e^{j \Delta s} \quad (\text{D.11})$$

between which $f_j(z)$ is approximated by a linear function. The value $\Delta s = 0.05$ was found to be small enough to give a good approximation to the relevant aerodynamic forcing function. These predefined

levels are also used to store output. The adaptive step size used for the numerical integration is always smaller than the distance between adjacent levels, so that several steps are necessary to integrate eq. (D.7) from one level to the next. We may generate output for z values between the levels. We shall use the term 'station' for a z value where output is generated. Thus the set of 'levels' is a subset of the set of 'stations'.

$$\begin{aligned} \mathbf{b}(z_j) &= \mathbf{b}(z_{j-1}) + \Delta \mathbf{b}(z_j) \\ \Delta \mathbf{b}(z_j) &= \int_{z_{j-1}}^{z_j} \mathbf{Y}^\dagger(z) \mathbf{F}(z) dz, \end{aligned} \quad (\text{D.12})$$

where b and Δb are now vectors and the forcing $\mathbf{F}(z)$ is assumed to be a linear function of z between two levels z_{j-1} and z_j . Therefore, we only need to calculate the integral for the special cases $f_k(z) = 1$ and $f_k(z) = z$ and store the six results, representing the full three-dimensional forcing, in a look-up table. The solution for any linear $f_k(z)$ can then be obtained as a linear combination of these two spatial cases.

The problem with solutions increasing or decreasing exponentially still remains, however, and the Y matrix becomes more and more ill-conditioned as we move upwards, and at some point we will be unable to invert it. However, we can avoid this, if the condition $\mathbf{Y}^\dagger \mathbf{X} = \mathbf{b}$ is modified. Thus for any regular matrix $\mathbf{B}$ we have

$$\mathbf{Y}^\dagger \mathbf{X} = \mathbf{b} \iff \mathbf{B} \mathbf{Y}^\dagger \mathbf{X} = \mathbf{B} \mathbf{b} \iff (\mathbf{Y} \mathbf{B}^\dagger)^\dagger \mathbf{X} = \mathbf{B} \mathbf{b} \quad (\text{D.13})$$

Here we should choose $\mathbf{B}$ so that $\mathbf{Y} \mathbf{B}^\dagger$ is numerically well behaved. Arguably, unitary matrices are the most well behaved of all matrices, and they are also extremely easy to invert, hence we will assume that $\mathbf{Y} \mathbf{B}^\dagger$ is unitary. This can be achieved in many ways, but in our case the QR decomposition is the appropriate choice. Recall that the QR decomposition rewrites Y as $Y = QR$, where Q is unitary and R is upper triangular ($R_{ij} = 0$ for $i > j$). With this choice, we have $\mathbf{Q} = \mathbf{Y} \mathbf{B}^\dagger$ and $\mathbf{B} = (\mathbf{R}^\dagger)^{-1}$, where $R^\dagger$ is a lower triangular matrix and $\mathbf{B}$, it's inverse, is therefore also lower triangular. This means that the first three components of $\mathbf{B} \mathbf{b}$ are still known because they are linear combinations of the known values b_1, b_2 and b_3 without any interference from the unknown values b_4, b_5 and b_6 .

Now we can explain the modified tracing method. The goal to have a condition equation at each level of the form

$$\mathbf{Q}^\dagger(z_j) \mathbf{X}(z_j) = \tilde{\mathbf{b}}(z_j) \quad (\text{D.14})$$

where Q is unitary and known, $\tilde{b}_1, \tilde{b}_2$ and $\tilde{b}_3$ are also known, while $\tilde{b}_4, \tilde{b}_5, \tilde{b}_6$ and X are unknown. We have already decided that $\mathbf{Y}(z_0) = \mathbf{Y}_0$ is unitary, so we may set

$$\mathbf{Q}(z_0) = \mathbf{Y}_0 \quad \text{and} \quad \tilde{\mathbf{b}}(z_0) = \mathbf{b}(z_0) \quad (\text{D.15})$$

Next step is to calculate $\mathbf{Q}(z_j)$ assuming that $\mathbf{Q}(z_{j-1})$ is known, because then, by induction, we have $Q(z_j)$ for all levels we need. This is done by solving the equation

$$\frac{\partial}{\partial z} \tilde{\mathbf{Y}}(z) = \mathbf{M}(z) \tilde{\mathbf{Y}}(z) \quad (\text{D.16})$$

$$\tilde{\mathbf{Y}}(z_{j-1}) = \mathbf{Q}(z_{j-1}) \quad (\text{D.17})$$

Thus $\tilde{\mathbf{Y}}$ obeys the same equation as $\mathbf{Y}$, but with a different initial condition set at z_{j-1} rather than at z_0 . $Q(z_j)$ is then found from the QR decomposition of $\tilde{\mathbf{Y}}(z_j)$, viz.

$$\tilde{\mathbf{Y}}(z_j) = \mathbf{Q}(z_j) \mathbf{R}(z_j) \quad (\text{D.18})$$

We also have

$$\frac{\partial}{\partial z} \tilde{\mathbf{Y}}^\dagger \mathbf{X} = (\mathbf{M} \tilde{\mathbf{Y}})^\dagger \mathbf{X} + \tilde{\mathbf{Y}}^\dagger (\mathbf{A} \mathbf{X} + \mathbf{F}) = \tilde{\mathbf{Y}}^\dagger (\mathbf{M}^\dagger + \mathbf{A}) \mathbf{X} + \tilde{\mathbf{Y}}^\dagger \mathbf{F} = \tilde{\mathbf{Y}}^\dagger \mathbf{F} \quad (\text{D.19})$$

Integrating both sides from z_{j-1} to z_j yields

$$\tilde{\mathbf{Y}}^\dagger(z_j) \mathbf{X}(z_j) - \tilde{\mathbf{Y}}^\dagger(z_{j-1}) \mathbf{X}(z_{j-1}) = \mathbf{R}^\dagger \mathbf{Q}^\dagger(z_j) \mathbf{X}(z_j) - \tilde{\mathbf{b}}(z_{j-1}) = \int_{z_{j-1}}^{z_j} \tilde{\mathbf{Y}}^\dagger(z) \mathbf{F}(z) dz \quad (\text{D.20})$$

so that

$$\mathbf{Q}^\dagger(z_j) \mathbf{X}(z_j) = (\mathbf{R}^\dagger(z_j))^{-1} \left(\tilde{\mathbf{b}}(z_{j-1}) + \int_{z_{j-1}}^{z_j} \tilde{\mathbf{Y}}^\dagger(z) \mathbf{F}(z) dz \right) = \tilde{\mathbf{b}}(z_j) \quad (\text{D.21})$$

or

$$\tilde{\mathbf{b}}(z_{j-1}) = \mathbf{R}^\dagger(z_j) \tilde{\mathbf{b}}(z_j) - \Delta \tilde{\mathbf{b}}(z_j) \quad (\text{D.22})$$

where

$$\Delta \tilde{\mathbf{b}}(z_j) = \int_{z_{j-1}}^{z_j} \tilde{\mathbf{Y}}^\dagger(z) \mathbf{F}(z) dz \quad (\text{D.23})$$

We also have the relation

$$\mathbf{X}(z_j) = \mathbf{Q}(z_j) \tilde{\mathbf{b}}(z_j) \quad (\text{D.24})$$

but $\tilde{b}_4$, $\tilde{b}_5$ and $\tilde{b}_6$ need to be determined before it is useful.

A look-up table containing z_j , $\mathbf{Q}(z_j)$, $\mathbf{R}^\dagger(z_j)$ and $\Delta \tilde{\mathbf{b}}(z_j)$ for all levels z_j , and sometimes also at points between levels, when there is a sign that $\tilde{\mathbf{Y}}$ is beginning to become ill-conditioned. This is not likely to happen when z_j is close to z_{j-1} , but higher up, where the distance between levels are larger, we sometimes get into troubles. An ill-conditioned, regular matrix is characterized by the so called condition number, which is the ratio between the largest and the smallest singular value. Recall that any matrix $\tilde{\mathbf{Y}}$ can be written as $\tilde{\mathbf{Y}} = \mathbf{U} \mathbf{D} \mathbf{V}$, where $\mathbf{U}$ and $\mathbf{V}$ are unitary $\mathbf{D}$ is diagonal (the singular value decomposition). The singular values are the absolute values of the diagonal element of $\mathbf{D}$. Thus $|\det \tilde{\mathbf{Y}}| = |\det \mathbf{D}|$ and $\|\tilde{\mathbf{Y}}\|^2 = \|\mathbf{D}\|^2$, where $\|\cdot\|$ is the Frobenius norm (some say 'Euclidian' norm. but in any case the rms value of the matrix elements). In addition we have

$$\begin{aligned} \det \tilde{\mathbf{Y}}(z+dz) &= \det \left\{ \tilde{\mathbf{Y}}(z) + \frac{d\tilde{\mathbf{Y}}(z)}{dz} dz \right\} = \det \tilde{\mathbf{Y}}(z) \det \left\{ \mathbf{1} + \tilde{\mathbf{Y}}^{-1}(z) \frac{d\tilde{\mathbf{Y}}(z)}{dz} dz \right\} \\ &= \det \tilde{\mathbf{Y}}(z) + \det \tilde{\mathbf{Y}}(z) \text{Tr} \left\{ \tilde{\mathbf{Y}}^{-1}(z) \frac{d\tilde{\mathbf{Y}}(z)}{dz} \right\} dz \end{aligned} \quad (\text{D.25})$$

so that

$$\frac{d \det \tilde{\mathbf{Y}}(z)}{dz} = \det \tilde{\mathbf{Y}}(z) \text{Tr} \left\{ \tilde{\mathbf{Y}}^{-1}(z) \frac{d\tilde{\mathbf{Y}}(z)}{dz} \right\} = \det \tilde{\mathbf{Y}}(z) \text{Tr} \mathbf{M}(z) = 0 \quad (\text{D.26})$$

since all diagonal elements of $\mathbf{M}$ are equal to 0. The initial value $\tilde{\mathbf{Y}}(z_{j-1})$ is unitary, hence $|\det \tilde{\mathbf{Y}}| = 1$ everywhere. The smallest singular value is therefore smaller than 1, while the largest singular value is larger than 1. A 'very' large value of $\|\tilde{\mathbf{Y}}\|^2$ is can therefore be taken as an indicator of $\tilde{\mathbf{Y}}$ becoming ill-conditioned. When the norm exceeds some preset value, an extra station is made with $\mathbf{QR}$ decomposition and storage in the look-up table.

The use of the look-up table is much as described above. First we decide a level where to put the upper boundary z_i (the i here is not a number, but stands for 'inversion'). We then determine $\mathbf{X}(z_i)$ from the 6 equations

$$\begin{aligned} X_j(z_i) &= 0 & \text{for } j=1,3 \text{ and } 5 \\ Q_{kj}^*(z_i) X_k(z_i) &= \tilde{b}_j & \text{for } j=1,2 \text{ and } 3 \end{aligned} \quad (\text{D.27})$$

Knowing $\mathbf{X}(z_i)$ we can find

$$\tilde{\mathbf{b}}(z_i) = \mathbf{Q}^\dagger(z_i) \mathbf{X}(z_i) \quad (\text{D.28})$$

Then we use (D.22) to find $\tilde{\mathbf{b}}$ at the station just below z_i and (D.24) to find $\mathbf{X}$ and continue downwards through the look-up table.

REFERENCES

Berezin, I. and Zhidkov, N. (1965). Chapter 9 - approximate methods of solving ordinary differential equations. In Berezin, I. and Zhidkov, N., editors, *Computing Methods*, pages 272–431. Pergamon, London.